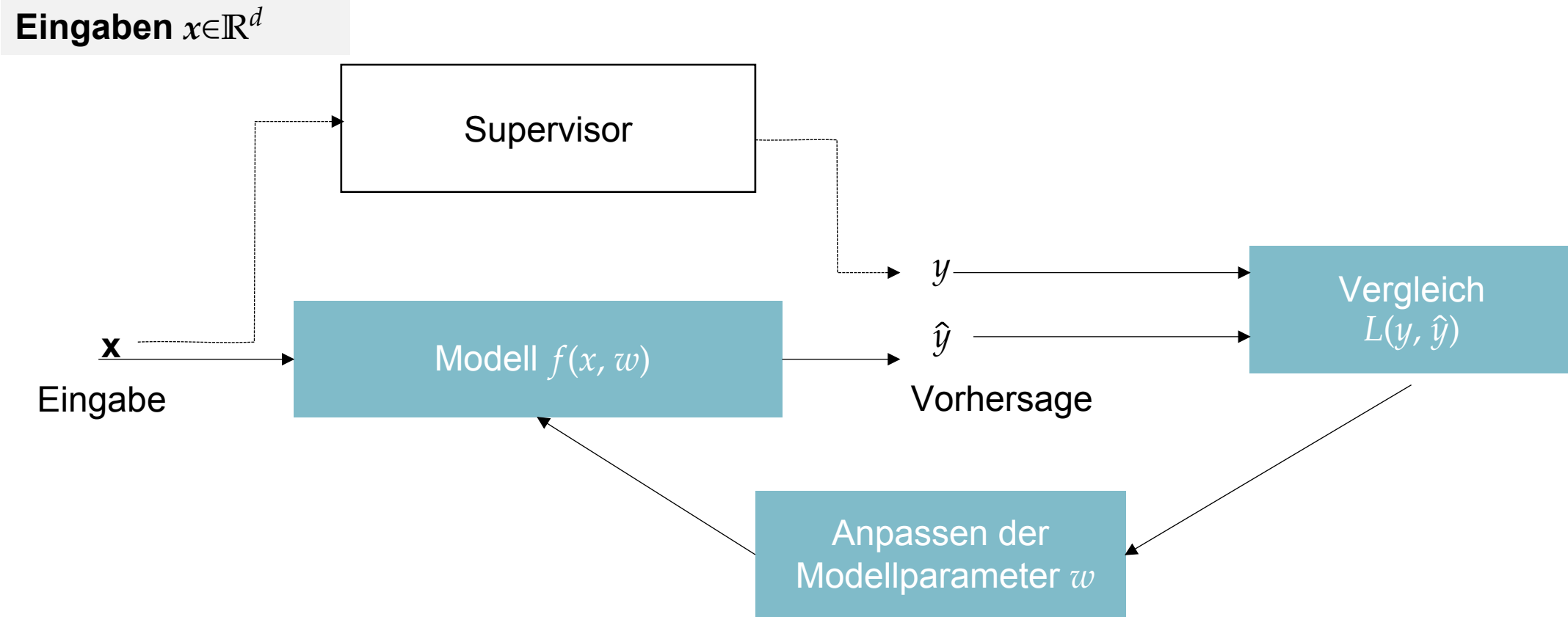


Datenanalyse in der Physik

Marcel Lüthi (D-INFK)
Vorlesung am D-Phys der ETH Zürich



Erinnerung: Überwachtes Lernen (supervised learning)



Erinnerung: Regression vs Klassifikation

Gegeben:

- Datenpaare $\{(x_1, y_1), \dots, (x_n, y_n)\}$ bestehend aus Merkmalen x_i und Zielvariable y_i

Regression: Zielvariable y variiert kontinuierlich ($y \in \mathbb{R}$)

Klassifikation: Zielvariable y kann nur diskrete Werte annehmen ($y \in \{0, \dots, k\}$)

Diskussion

- Überlegen Sie sich ein Beispiel eines Klassifikationsproblems aus Ihrem Alltag / Studium
 - Was sind die Merkmale?
 - Was ist die Zielvariable, die Sie vorhersagen möchten?

Erinnerung: Lineare Regression

Daten

$$D = \{(x_1, y_1), \dots, (x_n, y_n)\}, x_i \in \mathbb{R}, y_i \in \mathbb{R}$$

Modell

$$\hat{y} = f(x, w) = f(x, w_1, w_0) = w_1 x + w_0$$

Parameter

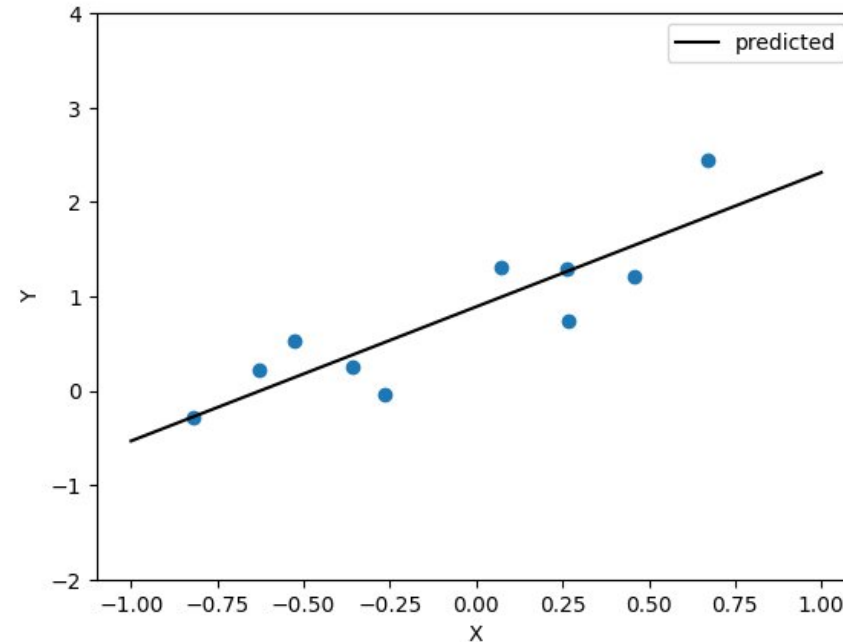
- Parameter vector $w = (w_0, w_1)^T$
- Steigung w_1 , Achsenabschnitt w_0

Verlustfunktion

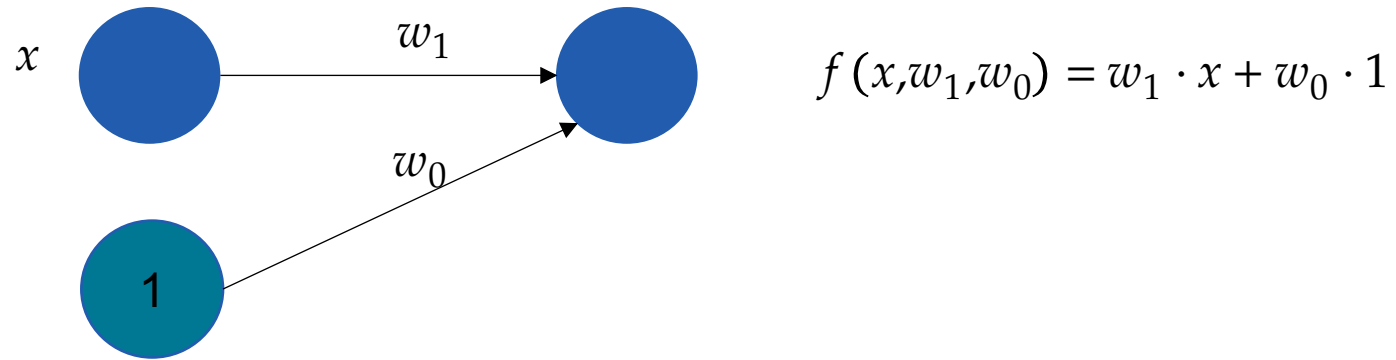
$$L(y, \hat{y}) = (y - f(x, w))^2$$

Anpassen der Parameter

- Optimierungsproblem $\min_w \sum_{i=1}^n (y_i - f(x_i, w))^2$
(Lösung durch Normalgleichung)



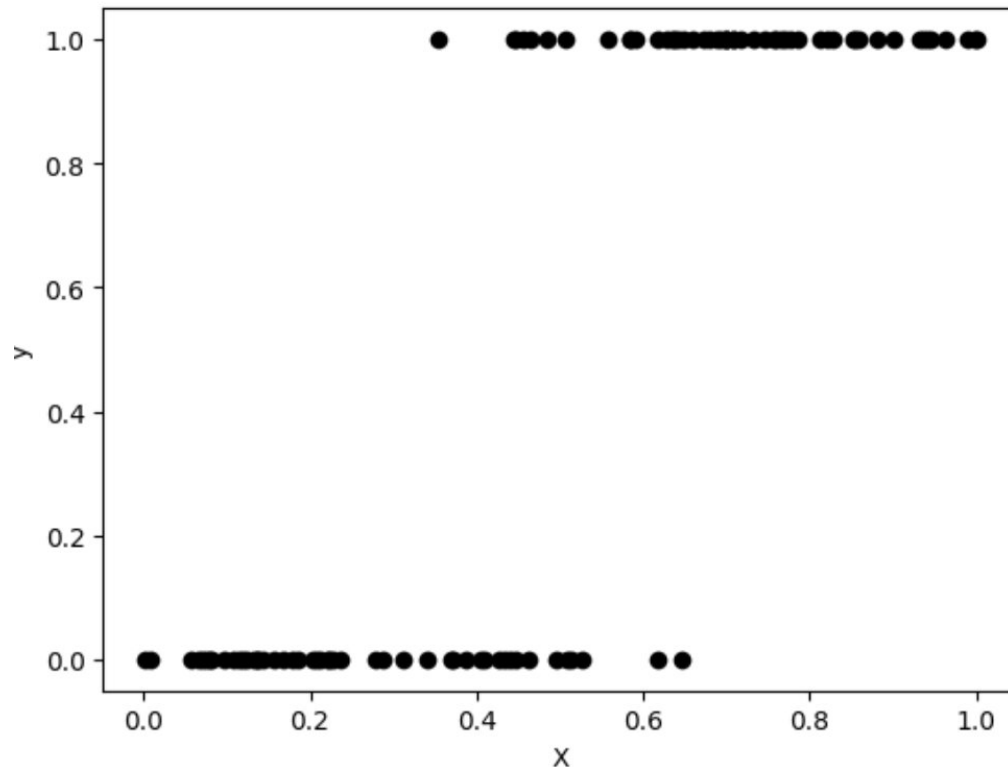
Erinnerung: Lineare Regression - Grafisches Modell



(Binäre) Klassifikation

Daten

$$D = \{(x_1, y_1), \dots, (x_n, y_n)\}, x_i \in \mathbb{R}, y_i \in \{0, 1\}$$



Ziel: Daten in zwei Klassen einteilen

Beispiele:

- Klassifikation eines Tumors in gutartig / bösartig (y) basierend auf Durchmesser des Tumors (x)
- Unterscheiden von Frauen und Männern (y) anhand der Schuhgrösse (x)

Mögliches Modell für die binäre Klassifikation

Modell

$$\hat{y} = f(x, w_1, w_0) = \sigma(w_1 \cdot x + w_0)$$

mit der *Logistischen Funktion*

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

Interpretation: Wahrscheinlichkeit, dass Merkmal x Klasse $Y = 1$ entspricht

$$P(Y = 1 \mid X = x) = \sigma(w_1 \cdot x + w_0)$$

Binäre Klassifikation durch Thresholding der Wahrscheinlichkeit

$$\hat{y} = \begin{cases} 0 & \text{falls } \hat{y} < 0.5 \\ 1 & \text{sonst} \end{cases}$$

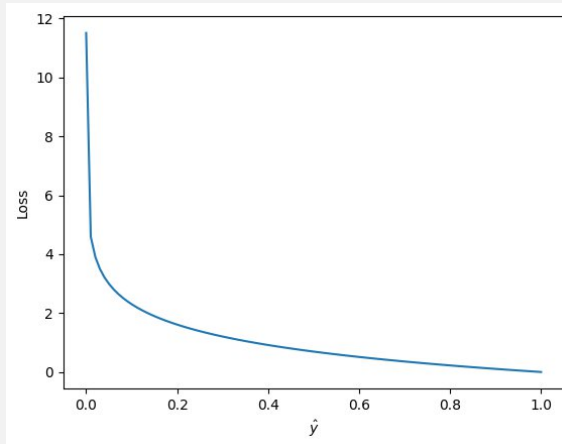
Mögliche Verlustfunktion für die Binäre Klassifikation

Verlustfunktion (Binary cross-entropy loss)

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

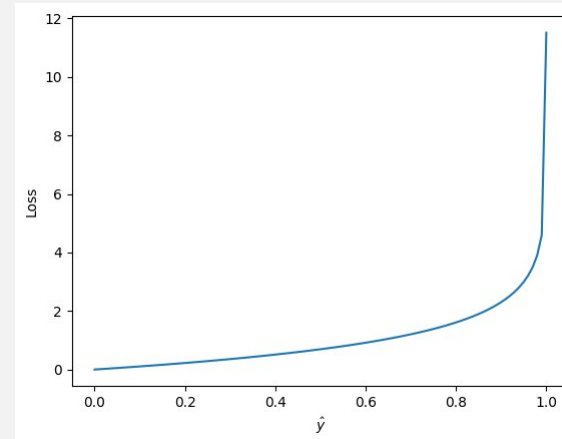
Case 1: $y = 1$

- Loss: $L(1, \hat{y}) = -1 \log(\hat{y})$



Case 2: $y = 0$:

- Loss: $L(0, \hat{y}) = -\log(1-\hat{y})$



- Verlust ist dann hoch, falls Vorhersage mit hoher Konfidenz falsch ist.

Logistische Regression für binäre Klassifikation

Modell

$$f(x, w_1, w_0) = \sigma(w_1 \cdot x + w_0)$$

mit der *Logistischen Funktion*

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

Parameter: Regressionskoeffizient w_1 und Bias w_0

Verlustfunktion (Binary cross-entropy loss)

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

mit

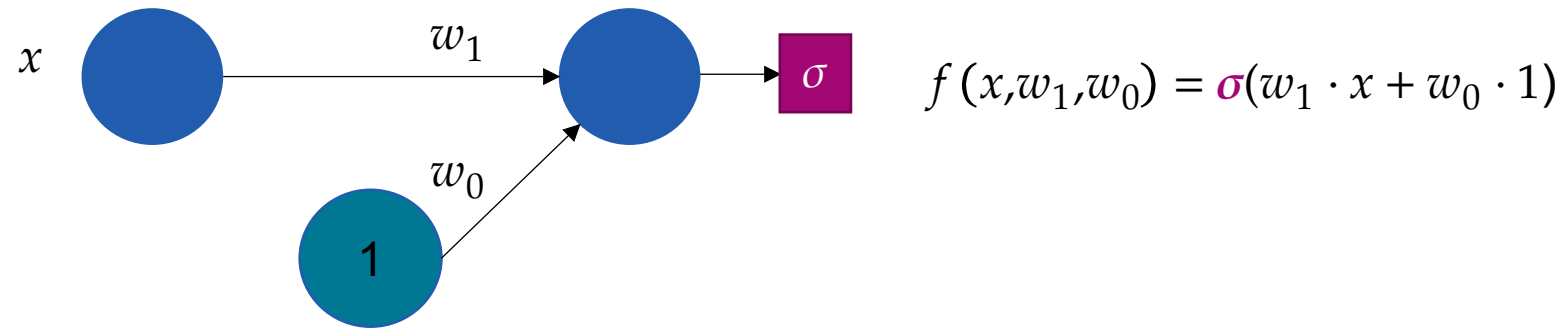
$$\hat{y} = f(x, w_1, w_0)$$

Optimierung

- Iterative Optimierung durch Gradientenabstieg

Berkson, Joseph. "Application of the logistic function to bio-assay." *Journal of the American statistical association* 39.227 (1944): 357-365.

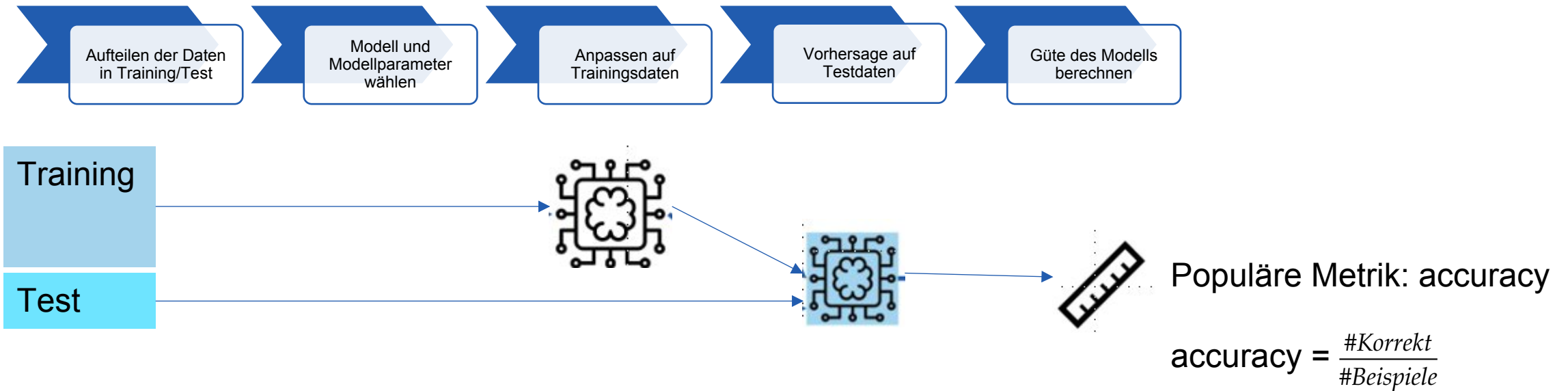
Logistische Regression: Grafisches Modell



Trainieren von Klassifikatoren

Prinzipien gleich wie bei Regression

- Metrik wird passend für Klassifikation gewählt.



Logistische Regression: Höhere Dimensionen

Modell

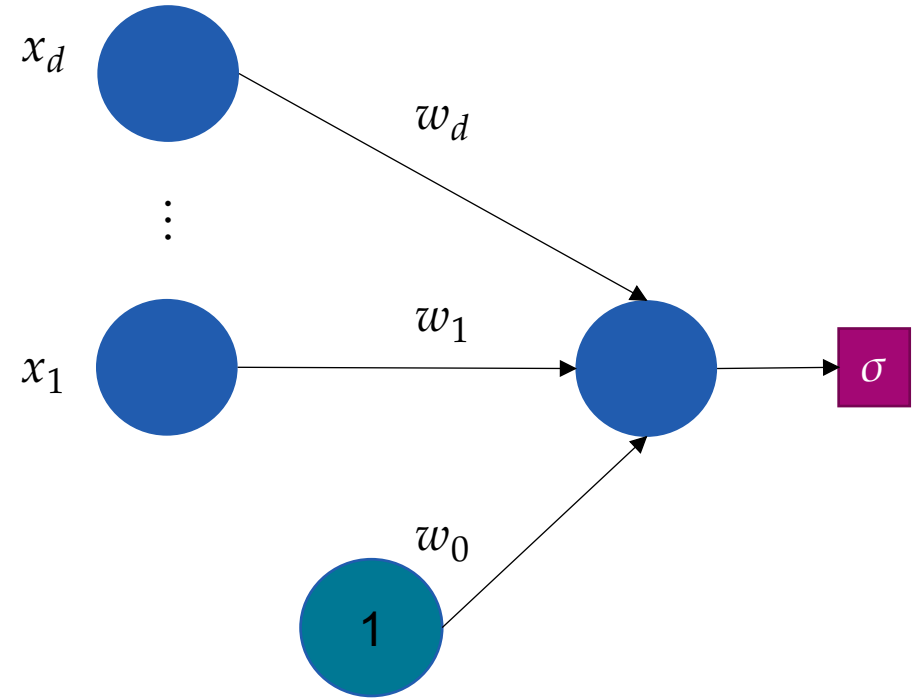
$$f(x, w) = \sigma \left(\sum_{i=1}^d w_i x_i + w_0 \right)$$

mit der *Logistischen Funktion*

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

Verlustfunktion

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$



Logistische Regression mit Scikit-learn

```
from sklearn.linear_model import LogisticRegression
```

```
# Apassen
```

```
model = LogisticRegression()  
model.fit(X, y)
```

```
# Vorhersagen
```

```
y_prob = model.predict_proba(X)[:,1]  
y_pred = model.predict(X)
```

Wert von $f(x, w)$
(interpretiert als $P(Y = 1|X = x)$)

Binäre Klassifikation:

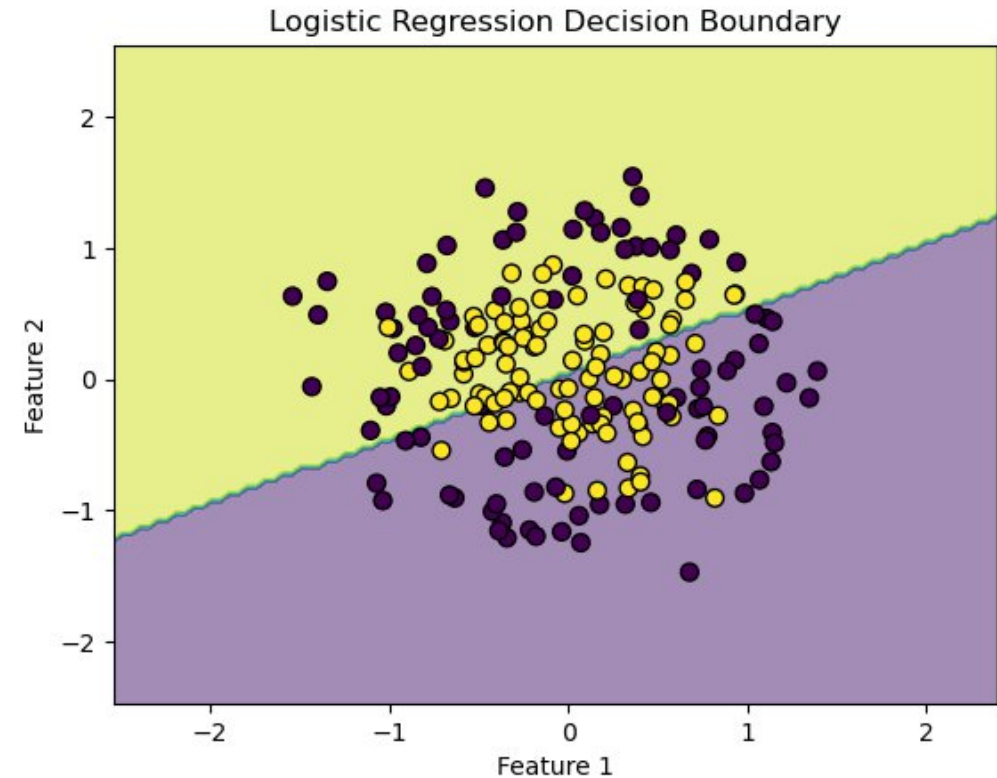
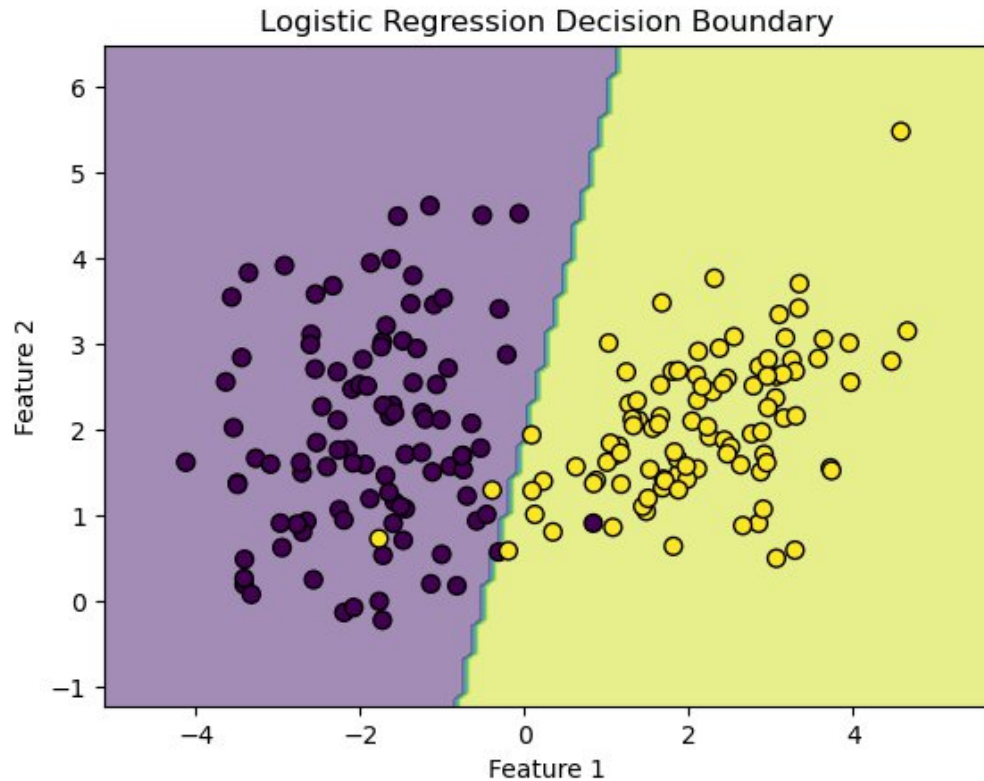
$$\begin{cases} 0 & \text{if } f(x, w) < 0.5 \\ 1 & \text{otherwise} \end{cases}$$



Lineare Basisfunktionsmodelle

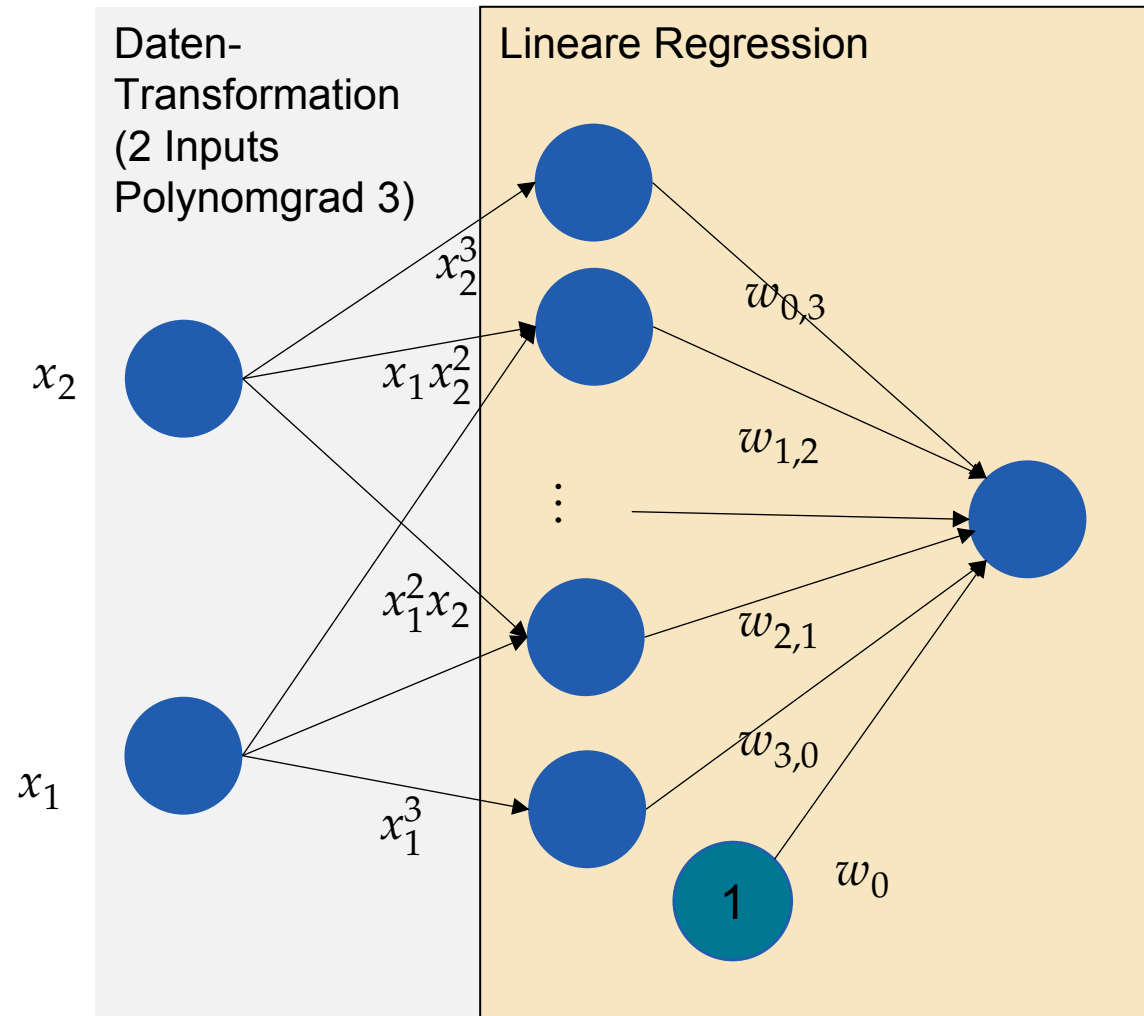
Einschränkung von Logistischer Regression

Entscheidungsgrenze ist immer linear!



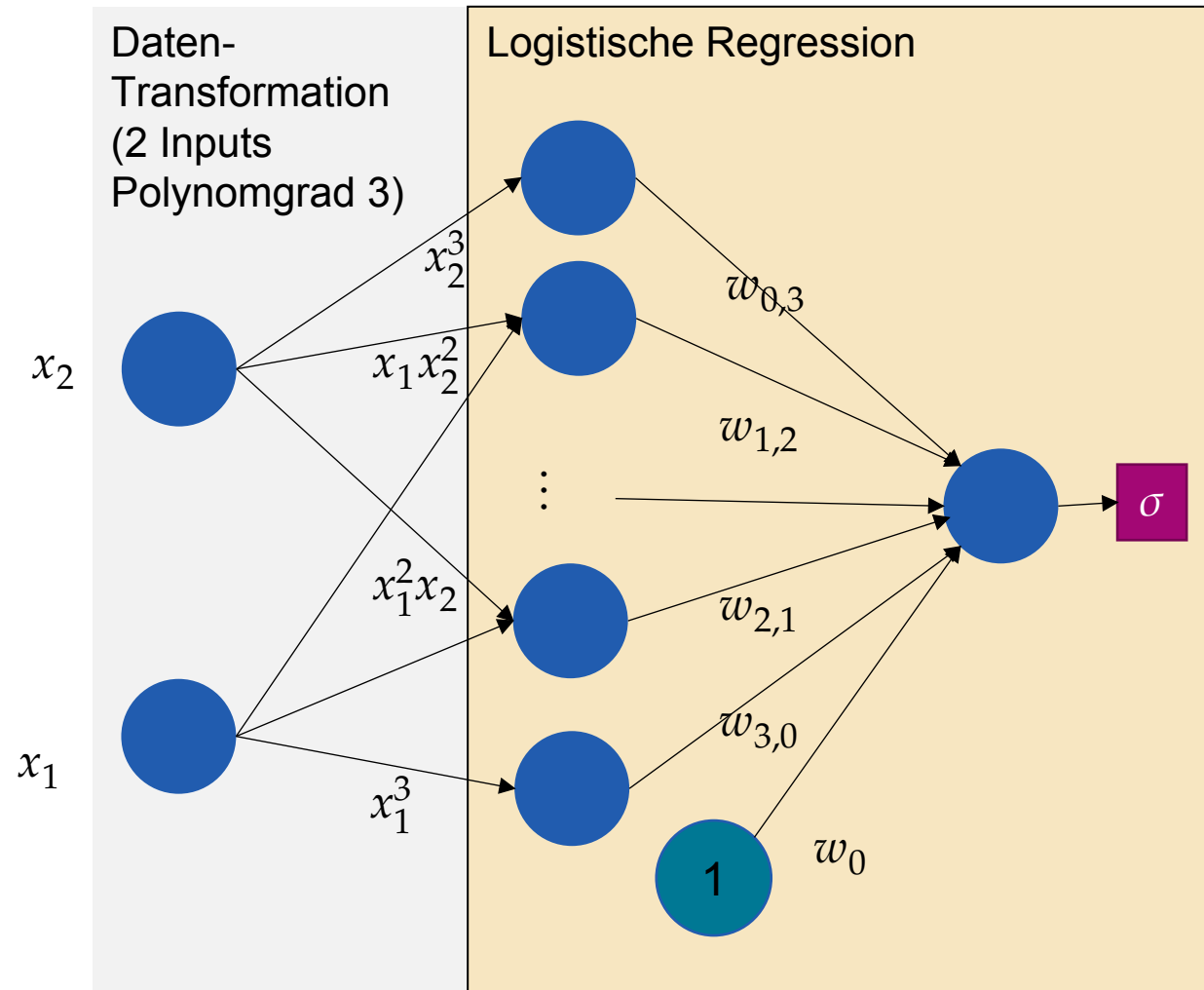
Wie könnten wir diese Einschränkung umgehen?

Erinnerung: Polynomielle Regression



$$f(x, w) = \sum_{k_1 + \dots + k_d \leq m} w_{k_1, \dots, k_d} x_1^{k_1} \dots x_d^{k_d}$$

Polynomielle logistische Regression



$$f(x, w) = \sigma \left(\sum_{k_1 + \dots + k_d \leq m} w_{k_1, \dots, k_d} x_1^{k_1} \dots x_d^{k_d} \right)$$

Trainieren einer Logistischen Regression mit Scikit-learn

```
# Erzeuge polynomiale Features (z.B. Grad 3)
```

```
poly = PolynomialFeatures(degree=3)
```

```
X_poly = poly.fit_transform(X)
```

```
# Trainiere logistische Regression auf den polynomialen Features
```

```
poly_model = LogisticRegression()
```

```
poly_model.fit(X_poly, y)
```

Verallgemeinerung: Lineare Basisfunktionsmodelle

Basisfunktionen

$$\phi_1, \dots, \phi_m, \phi_i: \mathbb{R}^d \rightarrow \mathbb{R}$$

Transformationsfunktion

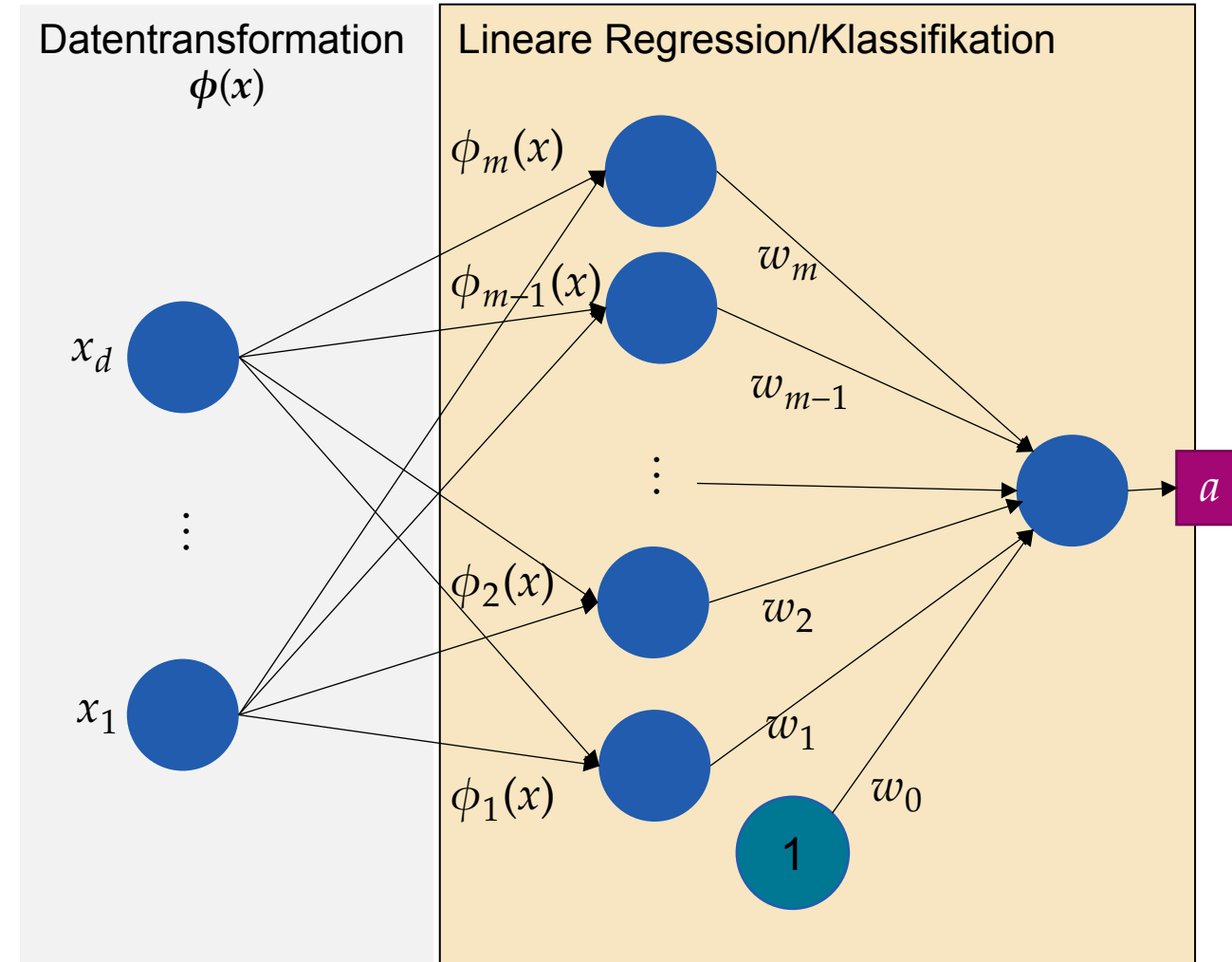
$$\phi(x) = (\phi_1(x), \dots, \phi_m(x))^T: \mathbb{R}^d \rightarrow \mathbb{R}^m$$

Model

$$f(x, w) = a(w_1 \phi_1(x) + \dots + w_m \phi_m(x) + w_0)$$

Aktivierungsfunktion

- Regression: $a(x) = x$
- Klassifikation: $a(x) = \sigma(x) = \frac{1}{1 + \exp(-x)}$



Lineare/logistische Basisfunktionsmodelle mit scikit-learn

Transformationsfunktion

```
def basisTransform(x):  
    return np.hstack([x, np.sin(3 * x), np.cos(3 * x)])
```

Anwender der Transformation

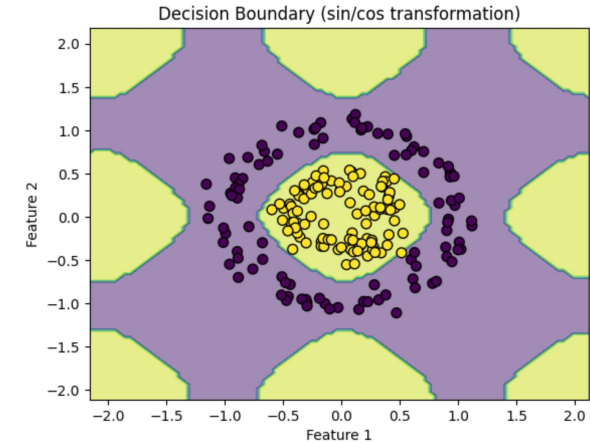
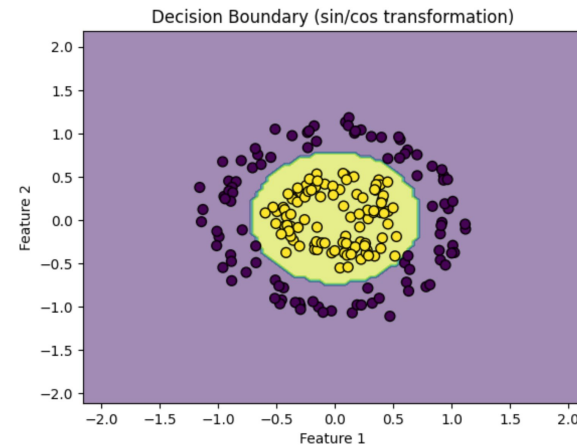
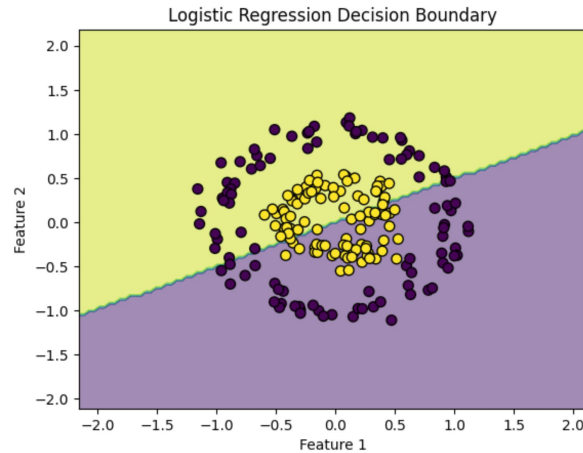
```
transformer = FunctionTransformer(basisTransform)  
X_transformed = transformer.fit_transform(X)
```

Erstellen und fitten des Modells

```
model = LogisticRegression()  
model.fit(X_transformed, y)
```



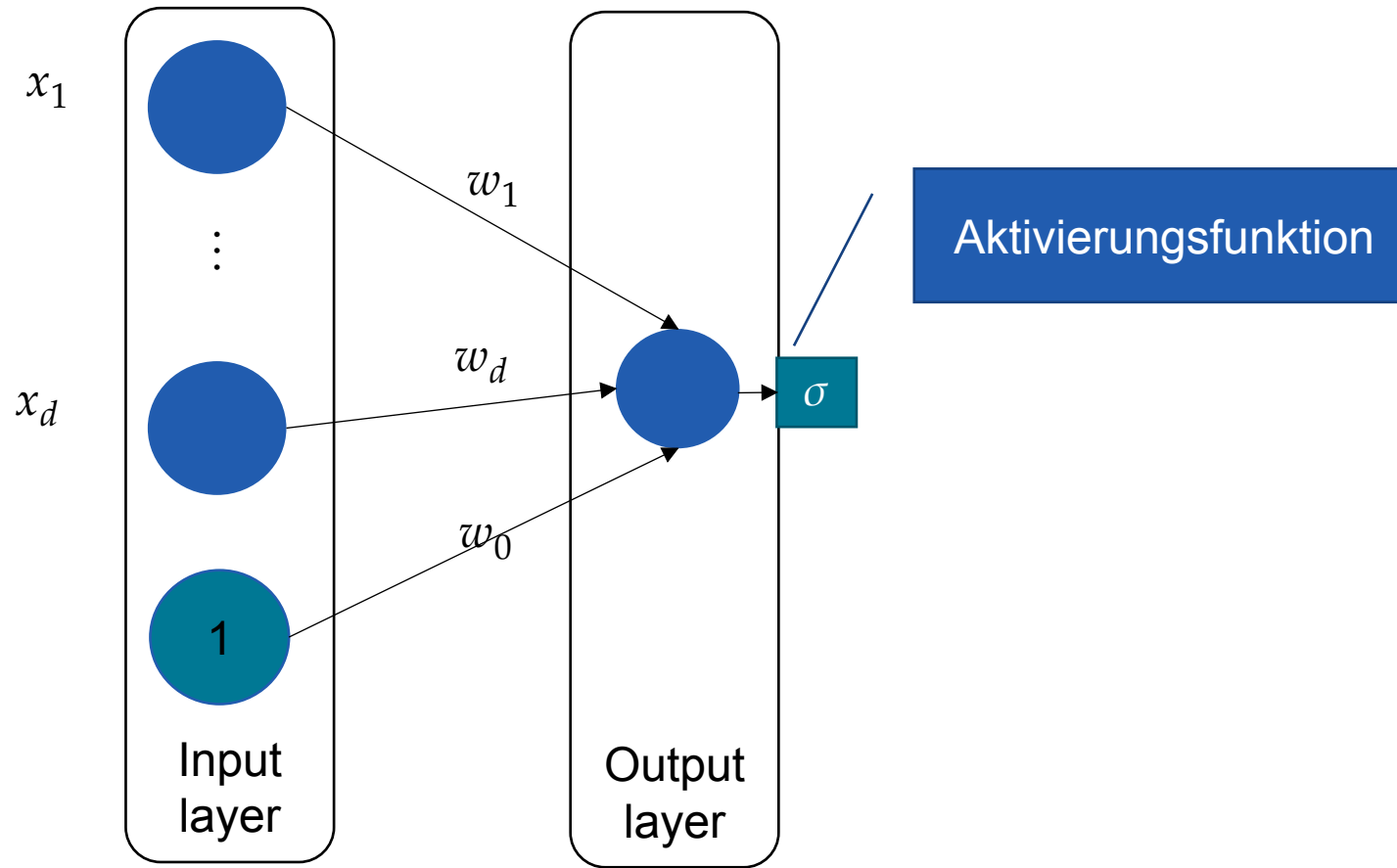
Diskussion: Wahl von Basisfunktionen



1. Gibt es eine “beste” Basisfunktion oder braucht es unterschiedliche Basisfunktionen für unterschiedliche Datensätze?
2. Welche Eigenschaften sollten gute Basisfunktion aufweisen?
3. Können Sie sich eine allgemeine Prozedur vorstellen um gute Basisfunktionen zu finden?

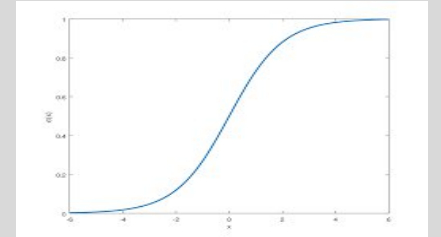
Neuronale Netze

Ein einfaches Neuronales Netz

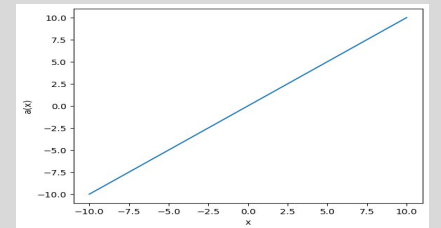


Aktivierungsfunktion

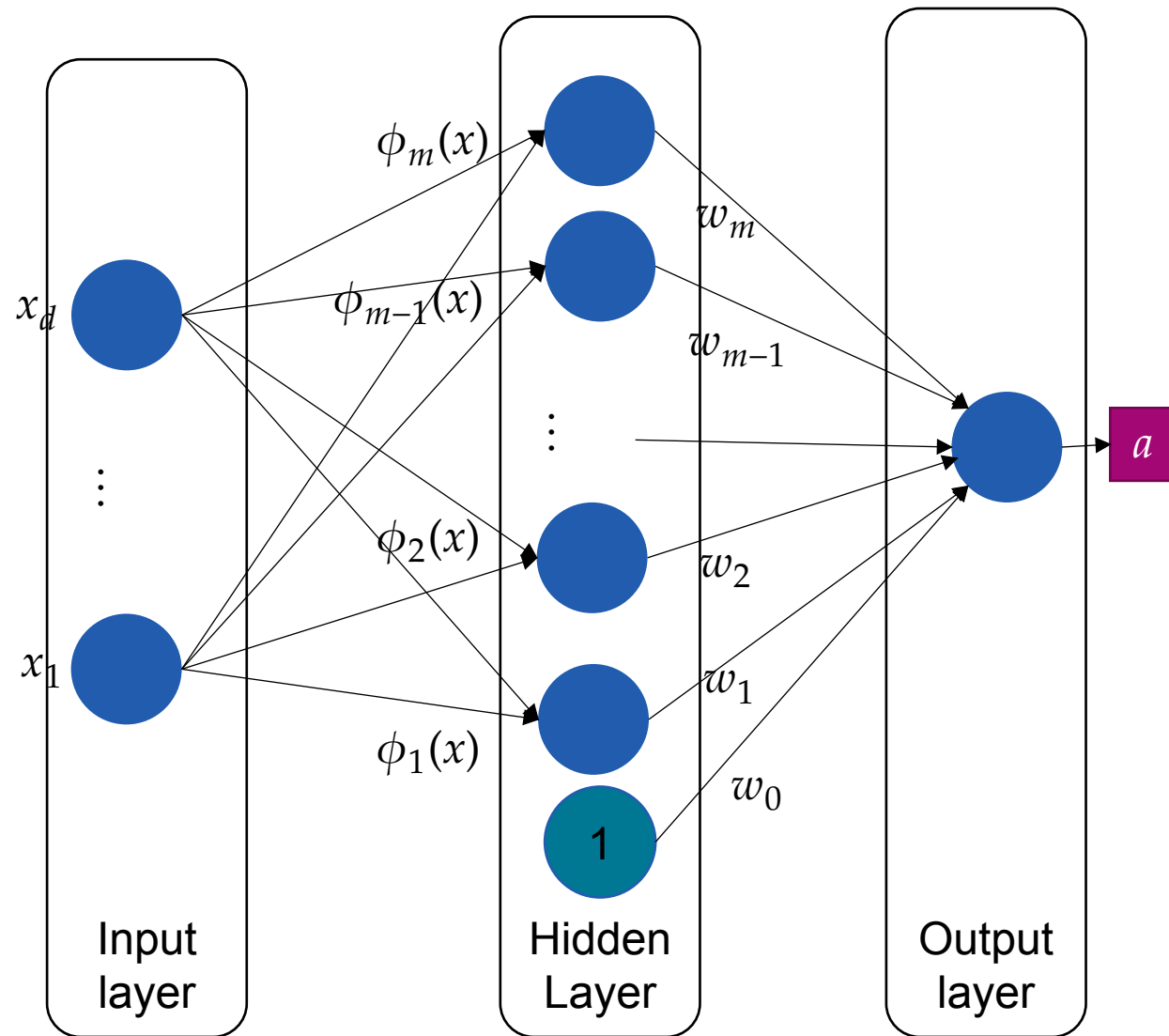
Sigmoid
(Binäre
Klassifikation)



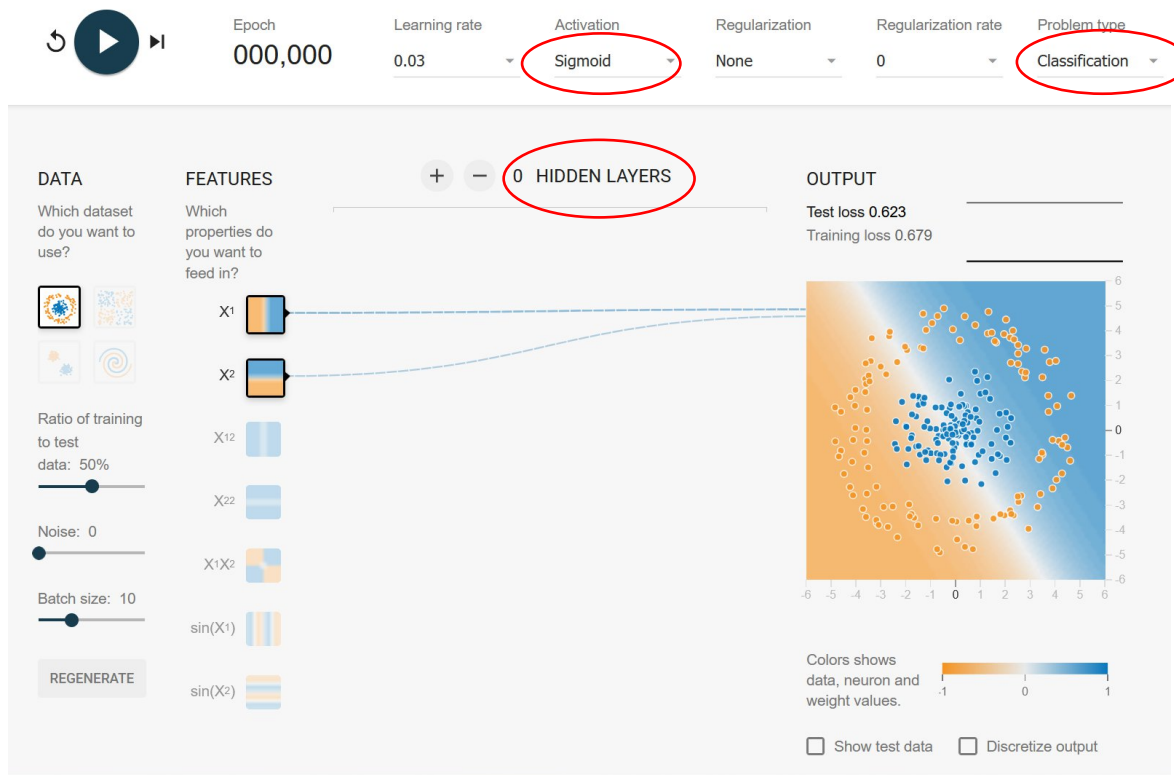
Identitätsfunktion
(Regression)



Ein komplexeres Neuronales netz



Neural network playground



Experimente:

Wählen Sie 0 hidden layers

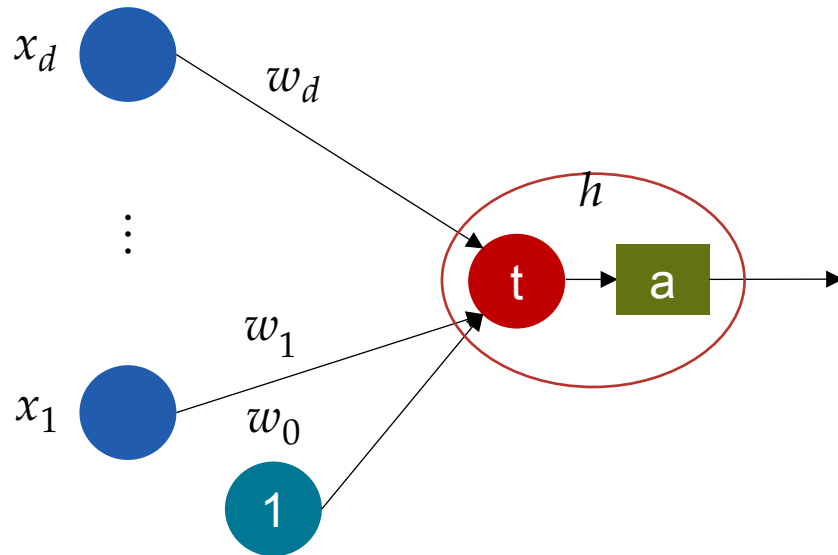
- Können Sie die Punkte korrekt klassifizieren?
- Mit welchen Features?
- Was passiert wenn Sie alle Feature wählen?
- Können Sie die Punkte der Spirale Klassifizieren?

<https://playground.tensorflow.org>

Neuronen: Lernbare Transformationen mit Nichtlinearität

Idee: Lerne lineare Transformation t gefolgt von nichtlinearer Aktivierungsfunktion a

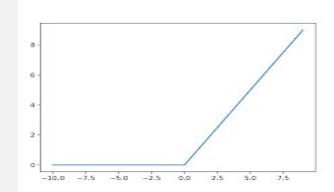
$$h(x, w) = a(t(x, w)) = a\left(\sum_{i=1}^d w_i x_i + w_0\right)$$



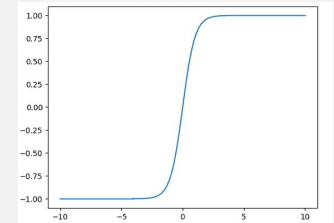
- Führt für jedes Neuron $d + 1$ neue Parameter ein

Einige Aktivierungsfunktionen

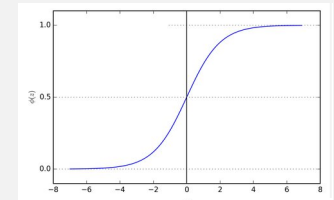
Relu



Tanh



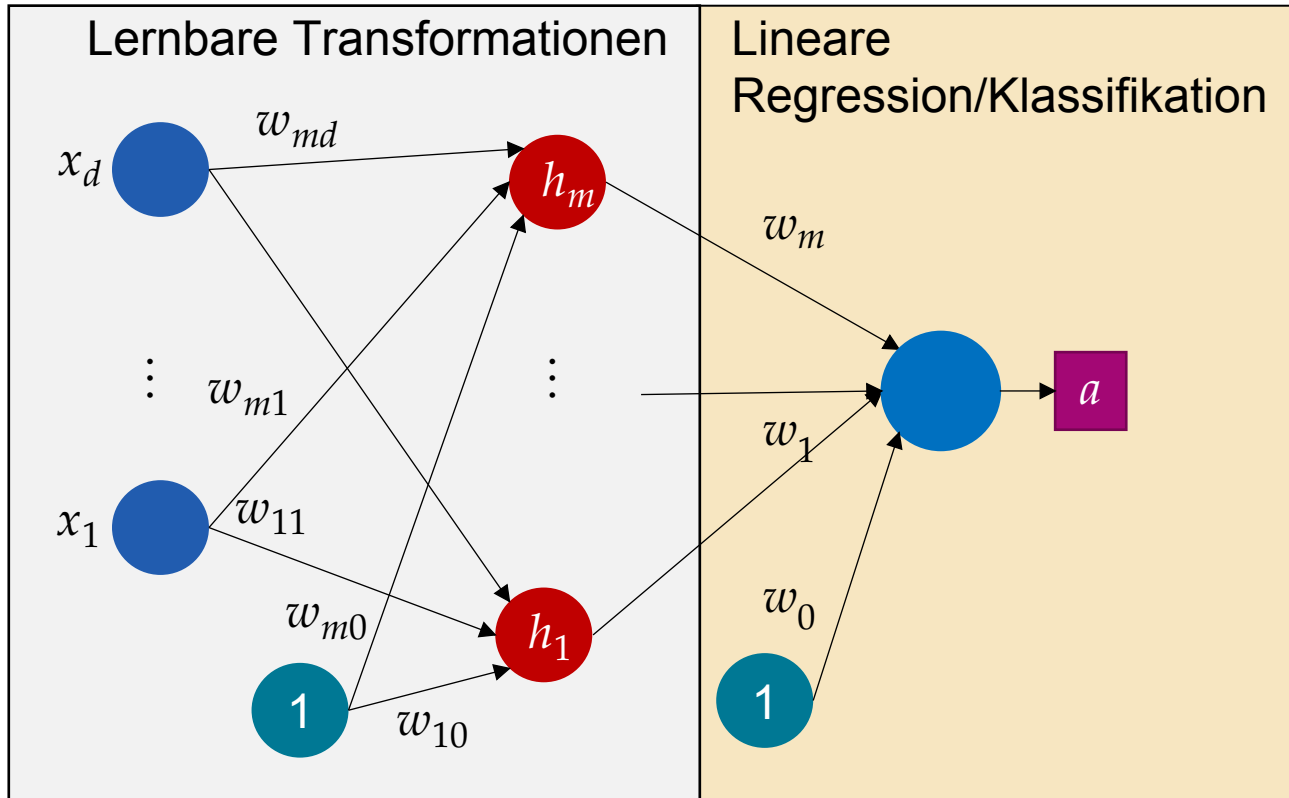
Logistic
(Sigmoid)



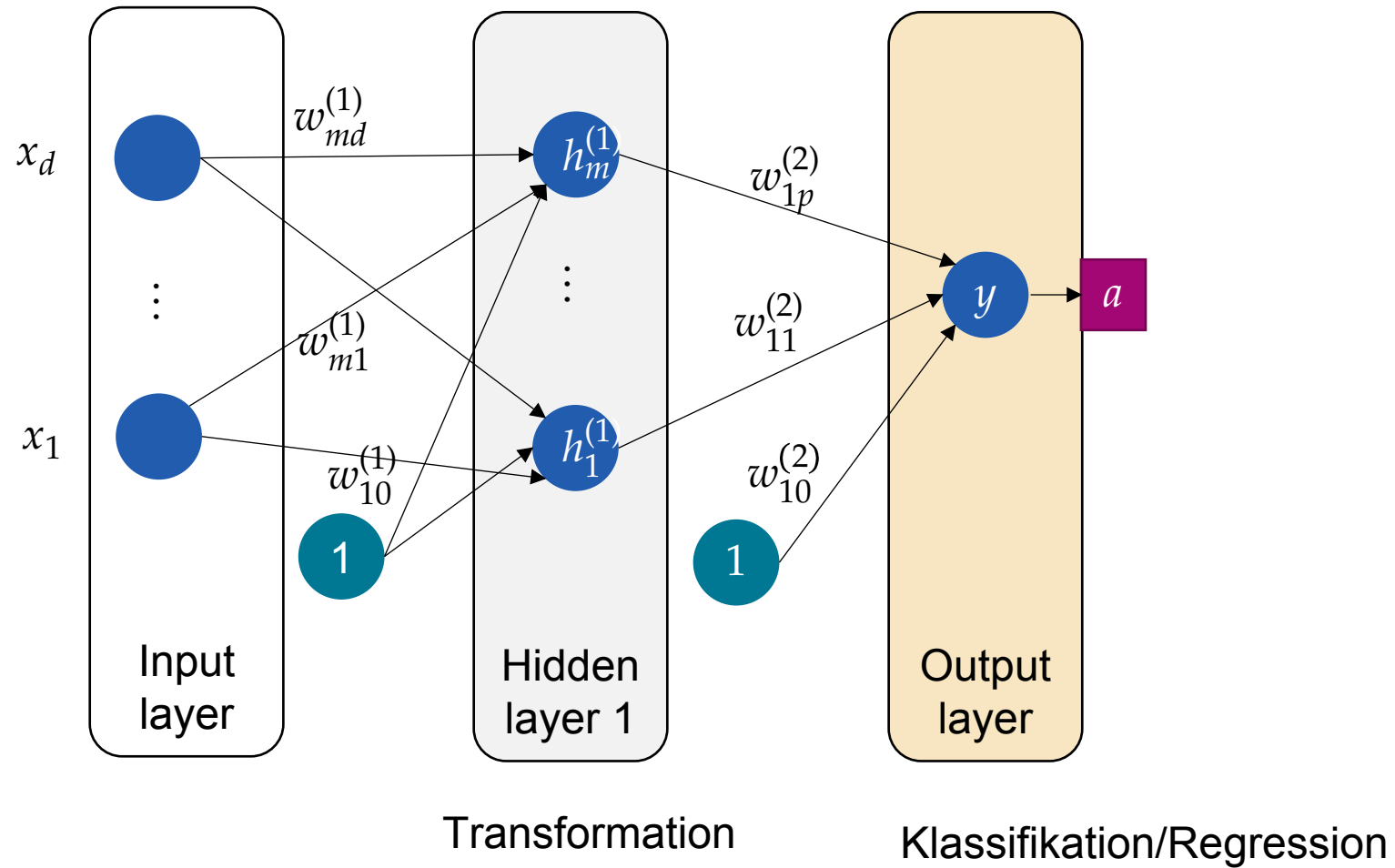
...

Netzwerke von lernbaren Transformationen

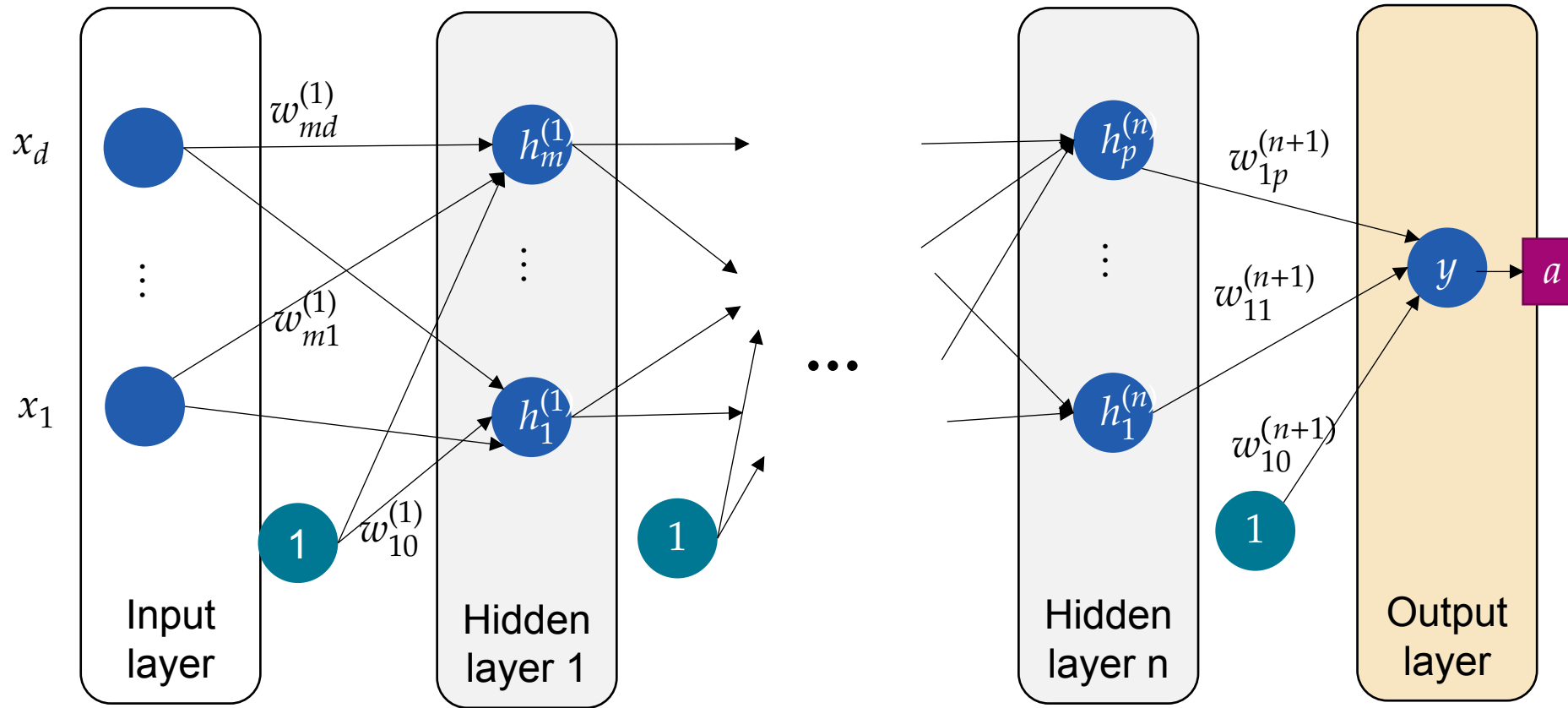
Wir können beliebig viele Transformationen lernen



Single-Layer Neuronales Netz



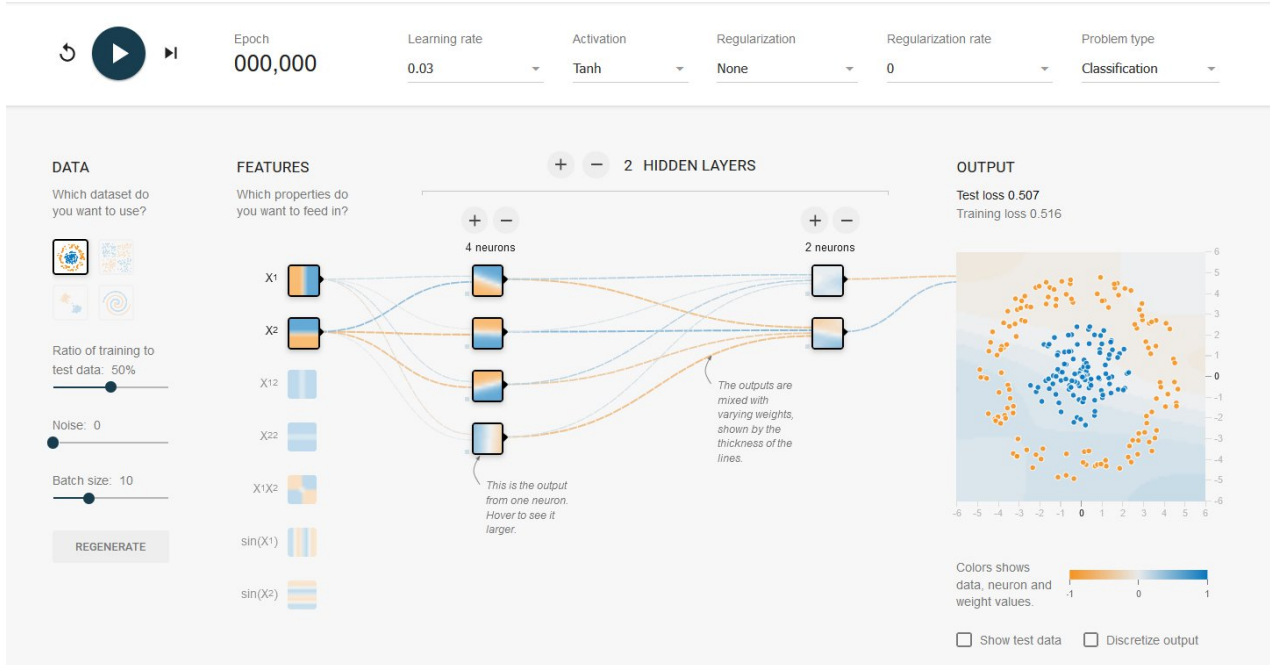
Multilayer neural networks



Terminologie

- Dense (oder Fully Connected): Jedes Neuron in Layer $i-1$ ist mit jedem Neuron in Layer i verbunden.

Neural network playground



Experimente:

Wählen Sie nun mehr als einen Hidden Layer

- Können Sie die Punkte der Spirale klassifizieren? Tip: Wählen Sie RELU als Aktivierungsfunktion)
- Beobachten Sie die Gewichte und Feature in den Layer

<https://playground.tensorflow.org>

Trainieren eines Multilayer Neuronalen Netz in scikit-learn

Klassifikation

```
mlp = MLPClassifier(hidden_layer_sizes=(50, 10), activation="relu", max_iter=10000)
mlp.fit(X, y)
y_pred_nn = mlp.predict(X)
```

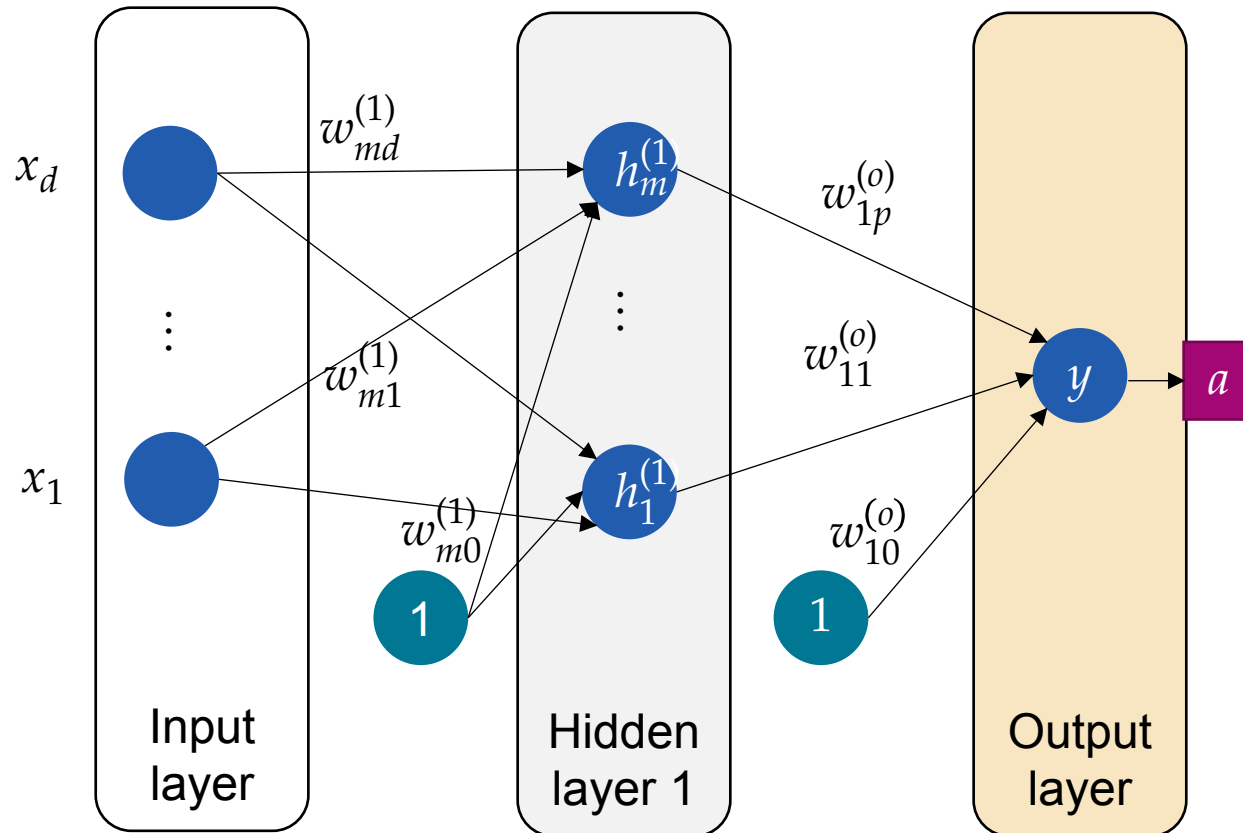
Regression

```
mlp = MLPRegression(hidden_layer_sizes=(50, 10), activation="relu", max_iter=10000)
mlp.fit(X, y)
y_pred_nn = mlp.predict(X)
```

Frage:

- Angenommen unsere Eingaben sind 10-Dimensional ($x \in \mathbb{R}^{10}$).
Wie viele Parameter haben dieses Netzwerk ungefähr?

Implementation von Neuronalen Netzen



Vorhersage (Detail siehe Übungsblatt):

- Lineare Transformation entspricht Matrixmultiplikation
- Aktivierungsfunktion wird Elementweise angewendet

$$h^{(1)} = a^{(1)}(W^{(1)}x)$$
$$y = a(W^o h^{(1)})$$

Training:

- Ableitung der Verlustfunktion nach den Parameter
 - Gradienten können automatisch bestimmt werden
- Optimierung mit Gradientenabstieg

Zusammenfassung

- Logistische Regression ist lineares Modell zur Klassifikation von Daten
 - Vorhersage der Klassenwahrscheinlichkeit $P(Y=1|X=x)$
 - Entspricht linearer Regression mit Sigmoid Funktion als "Aktivierungsfunktion"
- Ansatz für nichtlineare Klassifikation:
 1. Nichtlineare Transformation der Daten
 2. Anwendung einer linearen Klassifikation
- Neuronale Netze lernen Datentransformationen und lineare Klassifikation/Regression
 - Letzter Layer: Einfache logistische oder lineare Regression
- Deep Neural Networks
 - Mehrere (lernbare) nichtlineare Transformationen werden hintereinander ausgeführt
 - Daten werden schrittweise "linearisiert"